

MANUAL DE GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para desarrolladores en macOS con chips Apple Silicon (M1/M2/M3/M4), Zsh, Keychain y GitHub Actions

[CLI & Web Workflows](#)[SSH & GPG Signing](#)[GitHub Actions CI/CD](#)[Issues & Projects v2](#)[Security & Dependabot](#)[Packages & Releases](#)[API & Scripting](#)

Manual de GitHub: De Novato a Avanzado en macOS (Apple Silicon M1/M2/M3/M4)

Plataforma: macOS Sonoma / Sequoia / versiones modernas con arquitectura ARM64 (Apple Silicon)

Shell: Zsh (Z Shell por defecto de macOS)

Herramientas: Xcode Command Line Tools, Homebrew en `/opt/homebrew`, GitHub CLI (`gh`), Apple Keychain (`osxkeychain`), OpenSSH con Touch ID / Secure Enclave

Índice de Contenidos

1. [Parte I: Fundamentos y Configuración del Entorno en Apple Silicon](#)
 - 1.1 [Diferencias entre Git y GitHub](#)
 - 1.2 [Xcode CLI Tools y Homebrew ARM64 en `/opt/homebrew`](#)
 - 1.3 [Configuración de Identidad y Limpieza Global de `.DS\_Store`](#)
 - 1.4 [Integración con Apple Keychain \(`osxkeychain` \) y GitHub CLI](#)
 - 1.5 [Autenticación SSH Avanzada: Claves Ed25519 y Touch ID \(Secure Enclave\)](#)
 - 1.6 [Firma Criptográfica de Commits con SSH y GPG \(`pinentry-mac` \)](#)
2. [Parte II: Flujo de Trabajo Esencial \(Nivel Novato\)](#)
 - 2.1 [Creación y Clonación de Repositorios desde Terminal](#)
 - 2.2 [El Ciclo de Estados: Working Tree, Index y Commit en Zsh](#)
 - 2.3 [Staging Selectivo y Convención de Commits](#)
 - 2.4 [Sincronización con GitHub \(`push` , `pull` , `fetch` \)](#)
 - 2.5 [Control de Archivos Ignorados en macOS \(`.gitignore` \)](#)
3. [Parte III: Ramas, Fusiones y Estrategias Colaborativas \(Nivel Intermedio\)](#)
 - 3.1 [Gestión Eficiente de Ramas con `git switch` y Zsh](#)
 - 3.2 [Los Tres Métodos de Merge en GitHub](#)
 - 3.3 [Modelos de Flujo: GitHub Flow, Git Flow y Forking](#)
 - 3.4 [Operaciones de Reorganización: Stash, Cherry-Pick y Rebase Interactivo](#)
 - 3.5 [El Salvavidas: Recuperación con `git reflog`](#)
4. [Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo](#)
 - 4.1 [Tema 1: Prevención y Buenas Prácticas de Equipo](#)
 - 4.2 [Tema 2: Fusión Automática \(Cambios en Distintas Líneas\)](#)
 - 4.3 [Tema 3: Conflicto Directo de Fusión \(Mismas Líneas\) en macOS](#)
 - 4.4 [Tema 4: Elección Total de Versión \(`--ours` vs `--theirs` \)](#)

- 4.5 [Tema 5: Cambios Locales sin Confirmar al hacer Pull \(`git stash` \)](#)
 - 4.6 [Tema 6: Push Rechazado por Desfase \(`non-fast-forward` \) y Rebase Seguro](#)
 - 4.7 [Tema 7: Resolución de Conflictos en Pull Requests \(Web y CLI\)](#)
 - 4.8 [Tema 8: Conflicto de Modificación vs Eliminación](#)
 - 4.9 [Tema 9: Archivos Binarios y Bloqueo con Git LFS en macOS](#)
5. [Parte V: Gestión de Proyectos y Ecosistema GitHub](#)
- 5.1 [GitHub Issues, Hitos y Etiquetas desde Terminal](#)
 - 5.2 [Pull Requests y Revisiones de Código con GitHub CLI](#)
 - 5.3 [GitHub Projects \(v2\): Tableros y Automatización](#)
 - 5.4 [GitHub Discussions y Wikis Locales](#)
6. [Parte VI: Automatización y CI/CD con GitHub Actions \(Nivel Avanzado\)](#)
- 6.1 [Estructura de Workflows y Sintaxis YAML](#)
 - 6.2 [Pipelines con Runners Apple Silicon \(`macos-14` , `macos-15` \)](#)
 - 6.3 [Compilación Cruzada, Binarios Universales y Caching](#)
 - 6.4 [Configuración de un Self-Hosted Runner ARM64 como Launchd Daemon](#)
7. [Parte VII: Distribución, Paquetes y Publicación](#)
- 7.1 [GitHub Releases: Binarios \(`.dmg` , `.pkg` \) y Arquitecturas ARM64](#)
 - 7.2 [GitHub Packages y Distribución mediante Homebrew Taps](#)
 - 7.3 [GitHub Pages: Sitios Estáticos y Documentación](#)
8. [Parte VIII: Seguridad, Gobernanza y Políticas de Repositorio](#)
- 8.1 [Reglas de Protección de Ramas y Rulesets](#)
 - 8.2 [Dependabot, Secret Scanning y Push Protection](#)
 - 8.3 [Análisis Estático con CodeQL](#)
 - 8.4 [Gobernanza con `CODEOWNERS` y Permisos de Equipo](#)
9. [Parte IX: Scripting Zsh con la API y Diagnóstico](#)
- 9.1 [Consultas Avanzadas a la API con `gh api` y `jq`](#)
 - 9.2 [Automatización de Tareas con Scripts Zsh](#)
 - 9.3 [Diagnóstico y Resolución de Problemas Frecuentes en macOS](#)
-

Parte I: Fundamentos y Configuración del Entorno en Apple Silicon

1.1 Diferencias entre Git y GitHub

- **Git:** Sistema de control de versiones descentralizado optimizado para computación local de alto rendimiento. En macOS con procesadores Apple Silicon (M1/M2/M3/M4), Git aprovecha la arquitectura ARM64 para operaciones de árbol y cómputo de hashes ultrarrápidas.
- **GitHub:** Plataforma web y servicio de nube integral para la colaboración, revisión de código, flujos CI/CD con runners nativos de macOS, gobernanza corporativa y distribución de software.

1.2 Xcode CLI Tools y Homebrew ARM64 en `/opt/homebrew`

En Apple Silicon, Homebrew se instala en `/opt/homebrew` (en lugar de `/usr/local` de Intel), garantizando binarios nativos ARM64 sin emulación por Rosetta 2.

Paso 1: Instalar herramientas de línea de comandos de Apple

```
xcodes-select --install
```

¿Qué hace este comando?

Abre el instalador del sistema de macOS para descargar las cabeceras, compiladores (Clang) y utilidades de desarrollo necesarias de Xcode.

Paso 2: Instalar Homebrew oficial (si aún no lo tienes)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

¿Qué hace este comando?

Ejecuta el script oficial de instalación de Homebrew en la ruta nativa `/opt/homebrew`.

Paso 3: Configurar variables de entorno en Zsh

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

¿Qué hace este comando?

Añade las rutas de Homebrew a tu archivo `~/.zprofile` para que los binarios nativos de `/opt/homebrew/bin` se carguen con prioridad en cada sesión de Zsh.

Paso 4: Instalar las versiones nativas más recientes de Git, GitHub CLI y jq

```
brew install git gh jq
which git
file $(which git)
```

¿Qué hace este comando?

Instala Git, GitHub CLI y el procesador JSON `jq` y comprueba que el binario ejecutado sea un ejecutable Mach-O nativo ARM64.

1.3 Configuración de Identidad y Limpieza Global de `.DS_Store`

Finder en macOS crea automáticamente archivos ocultos `.DS_Store`. Debemos excluirlos globalmente.

Configurar nombre de autor y correo

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu-correo@ejemplo.com"
```

¿Qué hace este comando?

Establece la firma e identidad global para todas las confirmaciones locales.

Configurar finales de línea y exclusiones globales de macOS

```
git config --global core.autocrlf input
git config --global init.defaultBranch main

cat << 'EOF' > ~/.gitignore_global
.DS_Store
.AppleDouble
.LSOoverride
._*
.Spotlight-V100
.Trashes
EOF

git config --global core.excludesfile ~/.gitignore_global
```

¿Qué hace este comando?

Normaliza los saltos de línea a LF, fija `main` como rama inicial predeterminada y vincula un `.gitignore` global que impide que los archivos basura de macOS se mezclen en cualquier repositorio.

1.4 Integración con Apple Keychain (`osxkeychain`) y GitHub CLI

macOS almacena de forma nativa las credenciales seguras en el Llavero de Apple (*Keychain*), cifrado por hardware con el Secure Enclave:

```
git config --global credential.helper osxkeychain
gh auth login --hostname github.com --git-protocol ssh --web
```

¿Qué hace este comando?

Activa el asistente del llavero de Apple para Git e inicia sesión en GitHub CLI vinculando tus credenciales seguras en el navegador web.

```
gh auth status
```

¿Qué hace este comando?

Valida el estado de la conexión activa con GitHub y muestra los permisos del usuario actual.

1.5 Autenticación SSH Avanzada: Claves Ed25519 y Touch ID (Secure Enclave)

Opción A: Clave Ed25519 integrada con el Keychain de Apple

```
ssh-keygen -t ed25519 -C "tu-correo@ejemplo.com" -f ~/.ssh/id_ed25519

cat << 'EOF' >> ~/.ssh/config
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519
    AddKeysToAgent yes
    UseKeychain yes
EOF
chmod 600 ~/.ssh/config

ssh-add --apple-use-keychain ~/.ssh/id_ed25519
gh ssh-key add ~/.ssh/id_ed25519.pub --title "MacBook-AppleSilicon"
ssh -T git@github.com
```

¿Qué hace este comando?

Genera la clave Ed25519, configura `UseKeychain yes` para recordar la clave tras reiniciar macOS, la añade al llavero y la sube a GitHub mediante `gh`.

Opción B: Clave respaldada por hardware con Touch ID (FIDO2 / Secure Enclave)

```
ssh-keygen -t ed25519-sk -C "tu-correo@ejemplo.com"
```

¿Qué hace este comando?

Crea una clave criptográfica que exige que toques físicamente el sensor de Touch ID de tu Mac para autorizar cada operación con GitHub.

1.6 Firma Criptográfica de Commits con SSH y GPG (pinentry-mac)

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar in the top-right corner says "ZSH". The terminal contains four lines of text: "git config --global gpg.format ssh", "git config --global user.signingkey ~/.ssh/id_ed25519.pub", "git config --global commit.gpgsign true", and "git config --global tag.gpgsign true".

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

¿Qué hace este comando?

Habilita la firma criptográfica automática de cada commit utilizando tu clave pública SSH de macOS para obtener la insignia **Verified** en GitHub.

Parte II: Flujo de Trabajo Esencial (Nivel Novato)

2.1 Creación y Clonación de Repositorios desde Terminal

Caso A: Crear proyecto local y publicarlo en GitHub

```
mkdir -p ~/Developer/mi-proyecto-mac && cd ~/Developer/mi-proyecto-mac
git init
echo "# Proyecto macOS Apple Silicon" > README.md
git add README.md
git commit -m "docs: inicializar proyecto con README"
gh repo create mi-proyecto-mac --public --source=. --remote=origin --push
```

¿Qué hace este comando?

Crea la carpeta en `~/Developer`, inicia el repositorio, genera el `README.md`, hace el primer commit y crea el repositorio remoto público en GitHub vinculándolo y enviando los cambios de inmediato.

Caso B: Clonar repositorios

```
git clone git@github.com:usuario/mi-proyecto.git
```

¿Qué hace este comando?

Descarga la copia exacta del repositorio y todo su árbol de ramas a tu Mac vía SSH.

2.2 El Ciclo de Estados: Working Tree, Index y Commit en Zsh

```
git status -s
```

¿Qué hace este comando?

Muestra el estado del árbol de trabajo en formato resumido.

```
git diff
```

¿Qué hace este comando?

Compara las modificaciones en tus archivos de disco con el área de preparación (staging).

```
git diff --staged
```

¿Qué hace este comando?

Muestra los cambios exactos ya preparados en el staging que formarán parte del siguiente commit.

2.3 Staging Selectivo y Convención de Commits

```
git add -p main.swift
```

¿Qué hace este comando?

Permite preparar de forma interactiva fragmentos específicos de código línea por línea.

Estándar Conventional Commits

Tipo	Objetivo	Ejemplo
feat:	Nueva funcionalidad	<code>git commit -m "feat(core): aceleración neuronal con Apple Neural Engine"</code>
fix:	Corrección de fallos	<code>git commit -m "fix(ui): resolver desbordamiento en pantallas Retina"</code>
docs:	Documentación	<code>git commit -m "docs: actualizar requisitos para Apple Silicon"</code>
perf:	Optimización de rendimiento	<code>git commit -m "perf(metal): optimizar sombreadores de cómputo"</code>
refactor:	Refactorización de código	<code>git commit -m "refactor: migrar cierres a Swift Concurrency"</code>
chore:	Mantenimiento y configuración	<code>git commit -m "chore: actualizar dependencias en Brewfile"</code>

2.4 Sincronización con GitHub (push, pull, fetch)

```
git fetch origin
```

¿Qué hace este comando?

Descarga las ramas y commits nuevos de GitHub sin modificar tus archivos de trabajo.

```
git pull --rebase origin main
```

¿Qué hace este comando?

Descarga los commits del servidor e inserta tus cambios locales al final de la cadena de forma limpia y lineal.

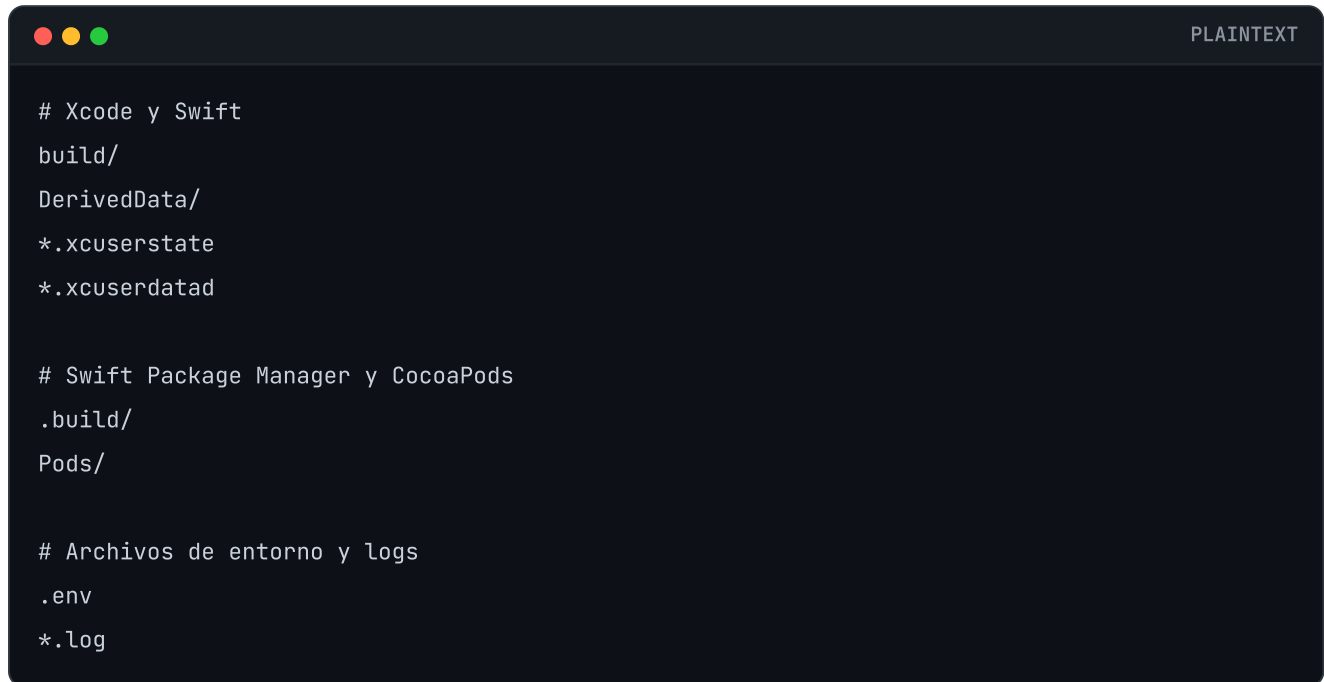
```
git push -u origin main
```

¿Qué hace este comando?

Sube tus commits locales a GitHub y configura el rastreo para los futuros `git push`.

2.5 Control de Archivos Ignorados en macOS (.gitignore)

Crea un archivo `.gitignore` con archivos generados por Xcode y Swift:



```
# Xcode y Swift
build/
DerivedData/
*.xcuserstate
*.xcuserdatad

# Swift Package Manager y CocoaPods
.build/
Pods/

# Archivos de entorno y logs
.env
*.log
```



```
git rm -r --cached build/
```

¿Qué hace este comando?

Remueve la carpeta `build/` del seguimiento de Git si fue subida por error, sin eliminarla del disco.

Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

3.1 Gestión Eficiente de Ramas con `git switch` y Zsh

```
git switch -c feature/procesamiento-metal
```

¿Qué hace este comando?

Crea una nueva rama de desarrollo llamada `feature/procesamiento-metal` y se posiciona en ella.

```
git push -u origin feature/procesamiento-metal
```

¿Qué hace este comando?

Publica la rama en GitHub y activa el seguimiento remoto.

```
git switch main
git branch -d feature/procesamiento-metal
git push origin --delete feature/procesamiento-metal
```

¿Qué hace este comando?

Regresa a `main`, borra la rama local ya fusionada y la elimina del servidor remoto en GitHub.

3.2 Los Tres Métodos de Merge en GitHub

- **Create a Merge Commit:** Une las ramas creando un commit conmemorativo que conserva el historial íntegro.
- **Squash and Merge:** Comprime todos los commits de la rama en un único commit descriptivo sobre la rama principal.
- **Rebase and Merge:** Añade los commits individualmente al final de la rama destino, logrando una historia completamente lineal.

3.3 Modelos de Flujo: GitHub Flow, Git Flow y Forking

Flujo de Trabajo con Forks en Proyectos Open Source

```
# 1. Crear el fork y clonarlo a tu Mac
gh repo fork swiftlang/swift --clone
cd swift

# 2. Mantener tu fork sincronizado con el repositorio original (upstream)
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

¿Qué hace este comando?

Crea una copia en tu cuenta, la clona a tu Mac y sincroniza tu copia con las últimas actualizaciones del proyecto original.

3.4 Operaciones de Reorganización: Stash, Cherry-Pick y Rebase Interactivo

```
git stash push -m "Optimizaciones de memoria en curso"
```

¿Qué hace este comando?

Guarda tus cambios no confirmados en la pila de stash con una etiqueta descriptiva.

```
git stash pop
```

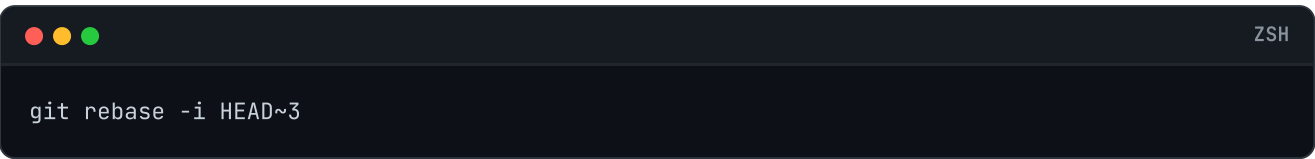
¿Qué hace este comando?

Recupera las modificaciones guardadas en el último stash y las vuelve a volcar en tus archivos.

```
git cherry-pick 7e3b12a
```

¿Qué hace este comando?

Extrae un commit específico de otra rama y lo aplica en la rama actual.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text "ZSH" is in the top-right corner. The command "git rebase -i HEAD~3" is entered in the terminal.

```
git rebase -i HEAD~3
```

¿Qué hace este comando?

Abre un editor interactivo en la terminal para editar, unir o reorganizar los últimos 3 commits.

3.5 El Salvavidas: Recuperación con `git reflog`

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text "ZSH" is in the top-right corner. The command "git reflog" is entered in the terminal.

```
git reflog
```

¿Qué hace este comando?

Inspecciona el historial cronológico de movimientos de `HEAD` en tu Mac.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text "ZSH" is in the top-right corner. The command "git reset --hard 'HEAD@{1}'" is entered in the terminal.

```
git reset --hard "HEAD@{1}"
```

¿Qué hace este comando?

Regresa de manera segura y exacta el repositorio al estado inmediatamente anterior al último error.

Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo

En el desarrollo profesional, es muy común que **dos personas toquen el mismo archivo en una misma carpeta al mismo tiempo**. Aquí se exponen las soluciones prácticas clasificadas por temas:

4.1 Tema 1: Prevención y Buenas Prácticas de Equipo

1. **Ramas independientes por tarea:** Prohibir commits directos sobre `main`. Trabajar siempre en ramas `feature/`.
2. **Modularización:** Dividir archivos gigantes en estructuras más pequeñas para aislar responsabilidades.
3. **Draft Pull Requests:** Publicar PRs en borrador para informar al equipo sobre qué archivos se están editando.
4. **Sincronizaciones periódicas:** Ejecutar `git pull --rebase origin main` al comenzar y finalizar cada jornada.

4.2 Tema 2: Fusión Automática (Cambios en Distintas Líneas)

Si tu compañero modificó una función al inicio del archivo y tú agregaste otra función al final, **Git es capaz de combinar ambas versiones automáticamente**:

```
# 1. Descargar e incorporar los cambios del compañero aplicando rebase
git pull --rebase origin main
```

¿Qué hace este comando?

Integra el commit del compañero y coloca tus commits encima sin generar conflicto manual alguno.

```
# 2. Subir el resultado integrado a GitHub
git push origin main
```

4.3 Tema 3: Conflicto Directo de Fusión (Mismas Líneas) en macOS

Si ambos modificaron exactamente la misma línea de código, Git marcará un conflicto. Configuraremos Visual Studio Code como herramienta visual de resolución:

Paso 1: Configurar VS Code como visor de diferencias y fusión en macOS

```
git config --global diff.tool vscode
git config --global difftool.vscode.cmd 'code --wait --diff $LOCAL $REMOTE'
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait $MERGED'
```

Paso 2: Ante un conflicto, abrir la herramienta visual

```
git mergetool
```

¿Qué hace este comando?

Abre automáticamente Visual Studio Code en macOS con el editor de 3 vías para aceptar cambios entrantes o locales con un clic.

Paso 3: Guardar y finalizar

- Si estabas en un merge:

```
git add src/App.swift
git commit -m "merge: resolver discrepancias de compilación en App.swift"
git push origin main
```

- Si estabas en un rebase:

```
git add src/App.swift
git rebase --continue
git push origin main
```

TIP

Para abortar y regresar al estado limpio previo:

```
git merge --abort
# 0 bien:
git rebase --abort
```

ZSH

4.4 Tema 4: Elección Total de Versión (--ours vs --theirs)

Cuando una de las dos versiones es 100% la correcta y la otra debe descartarse:

Conservar tu versión completa:

```
git checkout --ours src/App.swift
git add src/App.swift
git commit -m "resolve: conservar versión local de App.swift"
git push origin main
```

ZSH

Aceptar la versión del compañero por completo:

```
git checkout --theirs src/App.swift
git add src/App.swift
git commit -m "resolve: aceptar versión remota de App.swift"
git push origin main
```

ZSH

4.5 Tema 5: Cambios Locales sin Confirmar al hacer Pull (`git stash`)

Si tienes cambios locales sin confirmar en el archivo que tu compañero actualizó:

```
# 1. Guardar cambios en el stash de macOS
git stash push -m "Cambios locales en curso"

# 2. Descargar los cambios de GitHub
git pull --rebase origin main

# 3. Reaplicar tus cambios sobre la base nueva
git stash pop
```

¿Qué hace este comando?

Pone a salvo tus modificaciones en la pila temporal, descarga las novedades de GitHub y reaplica tus cambios.

4.6 Tema 6: Push Rechazado por Desfase (non-fast-forward) y Rebase Seguro

Si tu intento de `git push` es rechazado porque tu compañero se adelantó:

```
# 1. NUNCA ejecutes git push --force (borrarías el trabajo de tu compañero)

# 2. Sincroniza y sitúa tus commits por encima:
git pull --rebase origin main

# 3. Publica limpiamente:
git push origin main
```

4.7 Tema 7: Resolución de Conflictos en Pull Requests (Web y CLI)

Desde la Web de GitHub:

1. Abre el Pull Request.
2. Si existe colisión, haz clic en **Resolve conflicts**.
3. Elimina las marcas `<<<<<<` y `>>>>>>`, selecciona el código final y pulsa en **Commit merge**.

Desde la terminal con GitHub CLI (gh):

```

# Descargar la rama del PR
gh pr checkout 22

# Sincronizar con main para reproducir el conflicto
git merge origin/main

# Resolver en tu editor y confirmar
code src/App.swift
git add src/App.swift
git commit -m "merge: reconciliar con rama main"
git push origin HEAD

```

4.8 Tema 8: Conflicto de Modificación vs Eliminación

Ocurre cuando una persona editó el archivo y la otra lo borró del repositorio.

- Si deseas conservar el archivo:

```

git add src/App.swift
git commit -m "resolve: conservar archivo frente a eliminación"

```

- Si aceptas su eliminación:

```

git rm src/App.swift
git commit -m "resolve: confirmar eliminación del archivo"

```

4.9 Tema 9: Archivos Binarios y Bloqueo con Git LFS en macOS

Los archivos binarios (`.dmg` , `.zip` , `.psd` , modelos CoreML `.mlmodel`) no se pueden fusionar línea a línea.

```
# 1. Instalar Git LFS con Homebrew
brew install git-lfs && git lfs install

# 2. Configurar bloqueo para modelos de Machine Learning o archivos de diseño
git lfs track "*.mlmodel" --lockable
git add .gitattributes
git commit -m "chore: habilitar bloqueo exclusivo en modelos CoreML"
git push origin main

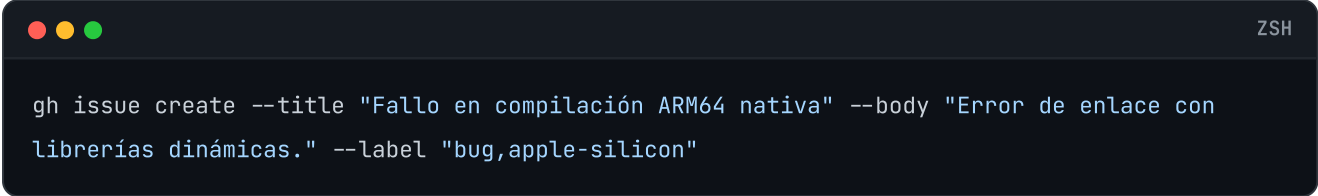
# 3. Bloquear un archivo antes de trabajar en él
git lfs lock Models/VisionModel.mlmodel

# 4. Ver bloqueos activos
git lfs locks

# 5. Desbloquear al terminar
git lfs unlock Models/VisionModel.mlmodel
```

Parte V: Gestión de Proyectos y Ecosistema GitHub

5.1 GitHub Issues, Hitos y Etiquetas desde Terminal



```
gh issue create --title "Fallo en compilación ARM64 nativa" --body "Error de enlace con librerías dinámicas." --label "bug,apple-silicon"
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The command is entered on a single line.

¿Qué hace este comando?

Registra una nueva incidencia con etiquetas en el repositorio remoto.



```
gh issue list --state open
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The command is entered on a single line.

¿Qué hace este comando?

Lista todas las incidencias abiertas actualmente en el proyecto.



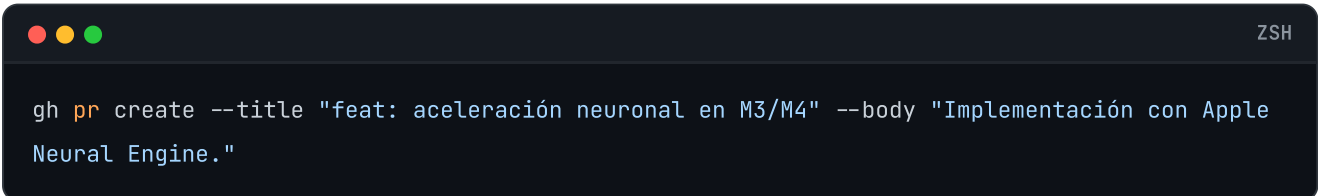
```
gh issue close 24 --reason "completed"
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The command is entered on a single line.

¿Qué hace este comando?

Cierra formalmente la incidencia #24 indicando que fue completada.

5.2 Pull Requests y Revisiones de Código con GitHub CLI



```
gh pr create --title "feat: aceleración neuronal en M3/M4" --body "Implementación con Apple Neural Engine."
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The command is entered on a single line.

¿Qué hace este comando?

Crea un Pull Request público para revisión del equipo.

```
gh pr checkout 9
```

¿Qué hace este comando?

Conmuta tu terminal a la rama del Pull Request #9 para probar los cambios en tu Mac.

```
gh pr review 9 --approve -b "Probado satisfactoriamente en M2 Max y M3 Pro."
```

¿Qué hace este comando?

Aprueba oficialmente los cambios del Pull Request con un mensaje de validación.

5.3 GitHub Projects (v2): Tableros y Automatización

```
gh project list  
gh project item-add 2 --owner "equipo-apple" --url  
"https://github.com/usuario/repo/issues/24"
```

¿Qué hace este comando?

Lista tus tableros y añade la incidencia #24 a la vista de tareas de tu proyecto.

5.4 GitHub Discussions y Wikis Locales

```
git clone git@github.com:usuario/mi-proyecto-mac.wiki.git  
cd mi-proyecto-mac.wiki  
echo "# Arquitectura Apple Silicon" > Arquitectura.md  
git add Arquitectura.md  
git commit -m "docs: documentar optimizaciones para SoC Apple"  
git push origin master
```

¿Qué hace este comando?

Clona la wiki como repositorio Git local, añade páginas en Markdown y las sube a GitHub.

Parte VI: Automatización y CI/CD con GitHub Actions (Nivel Avanzado)

6.1 Estructura de Workflows y Sintaxis YAML

Los workflows se crean bajo `.github/workflows/*.yaml` y se ejecutan ante eventos como `push` o `pull_request`.

6.2 Pipelines con Runners Apple Silicon (`macos-14`, `macos-15`)

GitHub incluye runners oficiales con chips Apple Silicon M1/M2/M3 bajo las etiquetas `macos-14` y `macos-15`.

Crea `.github/workflows/macos-ci.yaml`:

```
name: macOS Apple Silicon Native CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: macos-14 # Runner oficial con chip Apple Silicon (ARM64)

    steps:
      - name: Descargar repositorio
        uses: actions/checkout@v4

      - name: Seleccionar versión de Xcode
        run: sudo xcode-select -s /Applications/Xcode_15.4.app/Contents/Developer

      - name: Instalar dependencias mediante Homebrew
        run: |
          brew install cmake ninja

      - name: Compilar aplicación nativa Swift / C++
        run: |
          swift build -c release --triple arm64-apple-macosx

      - name: Ejecutar batería de pruebas
        run: |
          swift test

      - name: Subir binario compilado
        uses: actions/upload-artifact@v4
        with:
          name: app-macos-arm64
          path: .build/arm64-apple-macosx/release/
```

6.3 Compilación Cruzada, Binarios Universales y Caching

Utiliza la herramienta `lipo` de macOS para combinar binarios de Intel y Apple Silicon en un único ejecutable universal:

```
lipo -create -output BinarioUniversal Binario_arm64 Binario_x86_64
lipo -info BinarioUniversal
```

6.4 Configuración de un Self-Hosted Runner ARM64 como Launchd Daemon

Para usar un Mac mini o Mac Studio como servidor privado de compilación continuo:

```
# 1. Crear carpeta del runner
mkdir ~/actions-runner && cd ~/actions-runner

# 2. Descargar paquete nativo para macOS ARM64
curl -o actions-runner-osx-arm64-2.316.0.tar.gz -L
https://github.com/actions/runner/releases/download/v2.316.0/actions-runner-osx-arm64-
2.316.0.tar.gz
tar xzf ./actions-runner-osx-arm64-2.316.0.tar.gz

# 3. Configurar con el token del repositorio
./config.sh --url https://github.com/usuario/repo --token TU_TOKEN_DE_RUNNER

# 4. Instalar y arrancar como LaunchDaemon permanente en macOS
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

Parte VII: Distribución, Paquetes y Publicación

7.1 GitHub Releases: Binarios .dmg, .pkg y Arquitecturas ARM64



```

# 1. Crear y subir un tag firmado
git tag -a v1.5.0 -m "release: versión 1.5.0 nativa Apple Silicon"
git push origin v1.5.0

# 2. Publicar Release y adjuntar el instalador DMG
gh release create v1.5.0 ./build/MiApp-1.5.0-arm64.dmg \
  --title "MiApp v1.5.0 para macOS (Apple Silicon)" \
  --generate-notes

```

¿Qué hace este comando?

Publica la versión v1.5.0, genera notas automáticas y adjunta el archivo `.dmg` empaquetado para procesadores Apple Silicon.

7.2 GitHub Packages y Distribución mediante Homebrew Taps

Puedes distribuir tus binarios mediante un repositorio personal de Homebrew (Homebrew Tap) alojado en GitHub:

Crea un repositorio público llamado `homebrew-tap` y agrega la fórmula `Formula/miapp.rb`:

```
RUBY

class Miapp < Formula
  desc "Herramienta optimizada para Apple Silicon"
  homepage "https://github.com/usuario/miapp"
  url "https://github.com/usuario/miapp/releases/download/v1.5.0/miapp-1.5.0-arm64.tar.gz"
  sha256 "HASH_SHA256_DEL_ARCHIVO"
  version "1.5.0"

  def install
    bin.install "miapp"
  end
end
```

Los usuarios podrán instalarlo simplemente con:

```
ZSH

brew install usuario/tap/miapp
```

7.3 GitHub Pages: Sitios Estáticos y Documentación

Publica portales de documentación generados con frameworks modernos y desplégalos automáticamente mediante GitHub Actions.

Parte VIII: Seguridad, Gobernanza y Políticas de Repositorio

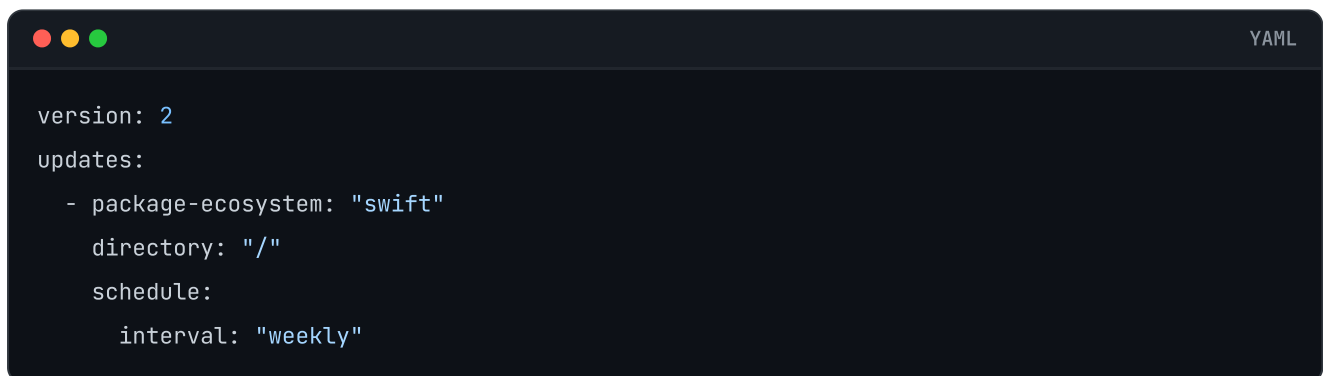
8.1 Reglas de Protección de Ramas y Rulesets

Protege la rama `main` en **Settings** → **Rules** → **Rulesets**:

- Prohibir commits directos sin Pull Request.
- Exigir que los pipelines de pruebas pasen satisfactoriamente.
- Exigir al menos una aprobación formal.
- Bloquear force-pushes (`git push --force`).

8.2 Dependabot, Secret Scanning y Push Protection

Configura `.github/dependabot.yml` para actualizar dependencias de Swift Package Manager o npm:



```
version: 2
updates:
  - package-ecosystem: "swift"
    directory: "/"
    schedule:
      interval: "weekly"
```

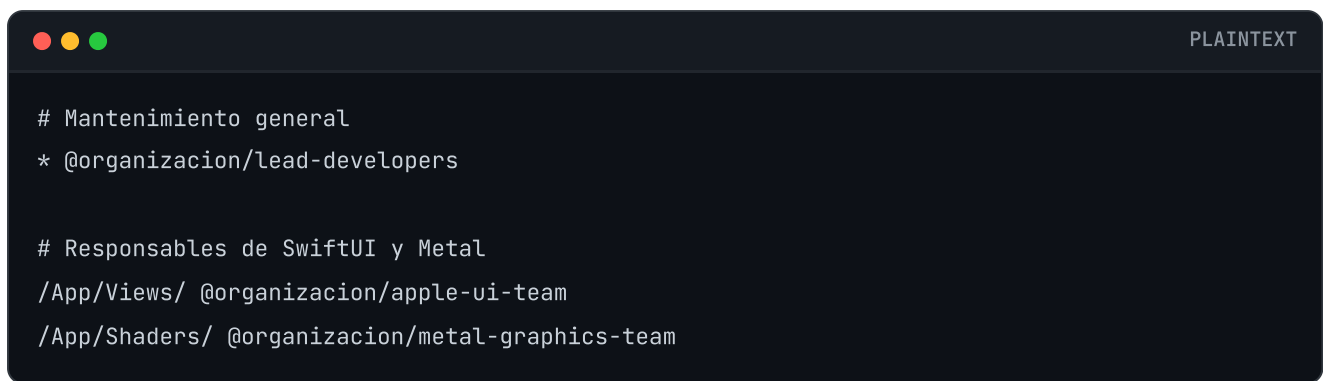
The screenshot shows a dark-themed code editor window with a title bar containing three colored circles (red, yellow, green) and the text "YAML". The editor displays a YAML configuration for Dependabot updates.

8.3 Análisis Estático con CodeQL

Inspecciona el código en cada PR en busca de fallos de memoria, vulnerabilidades de concurrencia y validaciones inseguras antes de fusionar.

8.4 Gobernanza con CODEOWNERS y Permisos de Equipo

Archivo `.github/CODEOWNERS` :

A dark-themed terminal window with a title bar at the top. The title bar contains three colored window control buttons (red, yellow, green) on the left and the word "PLAINTEXT" on the right. The terminal content is as follows:

```
# Mantenimiento general
* @organizacion/lead-developers

# Responsables de SwiftUI y Metal
/App/Views/ @organizacion/apple-ui-team
/App/Shaders/ @organizacion/metal-graphics-team
```

Parte IX: Scripting Zsh con la API y Diagnóstico

9.1 Consultas Avanzadas a la API con `gh api` y `jq`

```
gh api repos/:owner/:repo/pulls --jq '[] | {number: .number, title: .title, author: .user.login}'
```

¿Qué hace este comando?

Consulta los Pull Requests abiertos y extrae únicamente el número, título y autor en formato legible con `jq`.

9.2 Automatización de Tareas con Scripts Zsh

Script para sincronizar y purgar ramas obsoletas en macOS:

```
#!/usr/bin/env zsh
set -e

echo "Sincronizando y optimizando repositorio..."
git fetch -p origin
git remote prune origin

for branch in $(git branch --merged main | grep -v '^*' | grep -v 'main'); do
    git branch -d "$branch"
    echo "Rama local eliminada: $branch"
done

echo "Repositorio sincronizado y limpio."
```

9.3 Diagnóstico y Resolución de Problemas Frecuentes en macOS

1. Conflicto de versiones de Git (/usr/bin/git VS /opt/homebrew/bin/git)

- **Solución:** Asegura la precedencia de `/opt/homebrew/bin` en `~/.zprofile` :

```
echo 'export PATH="/opt/homebrew/bin:$PATH"' >> ~/.zprofile
source ~/.zprofile
```

2. Clave SSH no recordada tras reiniciar el Mac

- **Solución:**

```
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

3. Limpiar .DS_Store rastreados por error

- **Solución:**

```
find . -name .DS_Store -print0 | xargs -0 git rm --ignore-unmatch --cached
git commit -m "chore: remover archivos .DS_Store del repositorio"
```